

Language Frontend with LLVM Tutorial

AZIZA ZAMAN SADIA

ACM Reference Format:

Aziza Zaman Sadia. 2026. Language Frontend with LLVM Tutorial. 1, 1 (August 2026), 11 pages. <https://doi.org/XXXXXXX.XXXXXXX>

ID: 2231603642

Course: CSE425 - Concepts of Programming Languages

Section: 01

Instructor: Dr. Md Shahriar Karim [MSK1]

1 Introduction

The report depicts the iterative construction of a simple toy programming language called “Kaleidoscope”, using the LLVM framework to build its frontend from the ground up. It follows “My First Language Frontend” tutorial [1]. The primary goal is quite straight-forward - take the abstract theory from the course (lexical analysis, AST generation, IR compilation, and optimization passes) and actually make it run without losing our sanity in the process. Moreover, we highlight our own custom modifications and newly introduced language features - proving that theoretical grammar rules can, in fact, survive contact with real LLVM code.

2 Theory And Background

Building a compiler frontend requires translating raw, unstructured text into a representation that a computer can understand. This process depends on four fundamental phases of compiler theory covered in our course:

2.1 Lexical Analysis

Lexical analysis transforms an incoming stream of raw characters into a sequence of lexemes and tokens, discarding irrelevant characters like whitespace and comments.

- **Lexeme:** Lexemes are the lowest-level syntactic units or the actual character sequence in a source program that match a token pattern. For instance, in the statement `int sum = a + 25;`, the individual lexemes extracted are `'int'`, `'sum'`, `'='`, `'a'`, `'+'`, `'25'`, and `';'`.
- **Token:** Token is a category or classification assigned to a lexeme by the lexical analyzer. For the same example -
 - `'int'` → KEYWORD
 - `'sum'` → IDENTIFIER
 - `'='` → ASSIGNMENT_OF

Author's Contact Information: Aziza Zaman Sadia, aziza.sadia@northsouth.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, or post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2026/8-ART

<https://doi.org/XXXXXXX.XXXXXXX>

- 'a' → IDENTIFIER
- '+' → ADDITION_OP
- '25' → INTEGER
- ';' → SEMICOLON

Under the hood, lexical analysis relies on Regular Expressions (REs) to define what counts as a valid lexeme. These patterns compile into Deterministic Finite Automata (DFAs) which is essentially state-driven decision machines.

2.2 Syntax Analysis

After tokenization, the Syntax Analyzer evaluates token streams against formal structural constraints using a **Context-Free Grammar (CFG)**. These grammars are typically specified in **Backus-Naur Form (BNF)**—a metalanguage invented by John Backus and Peter Naur to document ALGOL 60, documenting the syntax rules of the compiler.

BNF rules rely on two fundamental types of symbols:

- **Terminal Symbols:** The concrete tokens and lexemes produced by the lexer (e.g., keywords, operators, and literals).
- **Non-Terminal Symbols:** Structural abstractions that represent syntactic categories (e.g., <expression> or <statement>), which can be expanded into multiple alternative definitions.

During parsing, the syntax analyzer matches the token stream against these BNF rules to demonstrate how input is derived from the start symbol, conceptually building a **Parse Tree**. To remove inefficiency, the compiler compresses this parse tree into an **Abstract Syntax Tree (AST)** by discarding non-terminal abstractions, punctuation. The AST acts as a main input for the later phases of compilation.

2.3 Intermediate Representation (IR) and Code Generation

Following the construction of the AST, the compiler takes the AST and translates into middle-tier code (IR).

A common IR style introduced in compiler theory is *Three-Address Code* (TAC). In TAC, each instruction has at most one operator and a maximum of three operands. Here, complex expressions are broken down into simple steps using temporary variables. For example, the expression $a = b * c + d$ can be rewritten as:

```
t1 = b * c
t2 = t1 + d
a = t2
```

We also introduce the idea of *Control Flow Graph* (CFG). It cuts TAC into chunks of basic blocks. Once compiler has this graph structure, it can perform optimization on it.

For LLVM, it uses something similar to TAC, but more structured. It also uses *Static Single Assignment* (SSA), where one variable can be used exactly once. By making dataflow relationships explicit, SSA simplifies critical analysis and optimization passes which makes LLVM IR an ideal compile target for the *Kaleidoscope* frontend.

2.4 Optimization Strategies on IR

After the compiler has translated the source code into IR, it can now apply different sorts of optimizations to improve performance. Different optimizations include:

- **Constant folding:** Here, the compiler evaluates the constant expression at the compiler time. It makes execution faster, as the math operations are completely removed from the final program. For example,

```
int seconds = 60*60*24;
```

it can be folded into `seconds = 86400;` directly in the IR.

- **Constant propagation:** If a variable is known to have a fixed constant value, the compiler replaces "all future references" to that variable with the literal constant value of itself. For instance:

```
x = 13;
y = x + 3;
```

it can be changed to `y = 16;`.

- **Copy propagation:** When one variable is assigned to another, then we can replace the use of assigned variable. For example:

```
y = 30 + 2;
x = y;
```

it can be propagated as `x = 32;`.

- **Dead code elimination:** Here, any unused segment of code is removed during compile time. For example,

```
x = useless_function();
y = 3;
return y;
```

in this code snippet, `x` is never read, the call to `useless_function()` is not necessary eliminated. This reduces the code size of the entire program resulting in a faster execution speed.

2.5 LLVM And The Kaleidoscope Language

The LLVM Project offers an SSA-based compiler infrastructure that separates front-end parsing and analysis from back-end machine code generation. Front-ends convert source programs into LLVM Intermediate Representation (IR), after which LLVM performs optimization and generates code for the target architecture. Within this framework, *Kaleidoscope* serves as a simple procedural language with implicitly typed 64-bit floating-point values, providing a practical and accessible example for understanding the fundamental stages of compiler construction.

3 Implementation Details

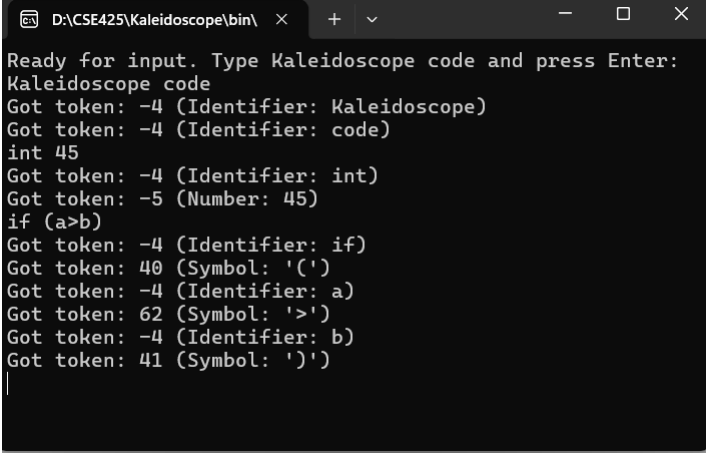
3.1 Lexer Design

```
enum Token {
    tok_eof = -1,
    tok_def = -2,
    tokExtern = -3,
    tok_identifier = -4,
    tok_number = -5,
};

static std::string IdentifierStr; // Filled in if tok_identifier
static double NumVal;           // Filled in if tok_number
```

Here, the lexer's job is to break raw text into discrete lexemes and tokens. This lexer right here reads one character at a time via `getchar()`, skipping whitespaces. Afterwards, it classifies identifiers, keywords, numeric literals, and any other character is returned verbatim as its ASCII value suggests.

Moreover, an incorrect or incomplete lexer would corrupt the entire compilation pipeline before it can even begin.



```

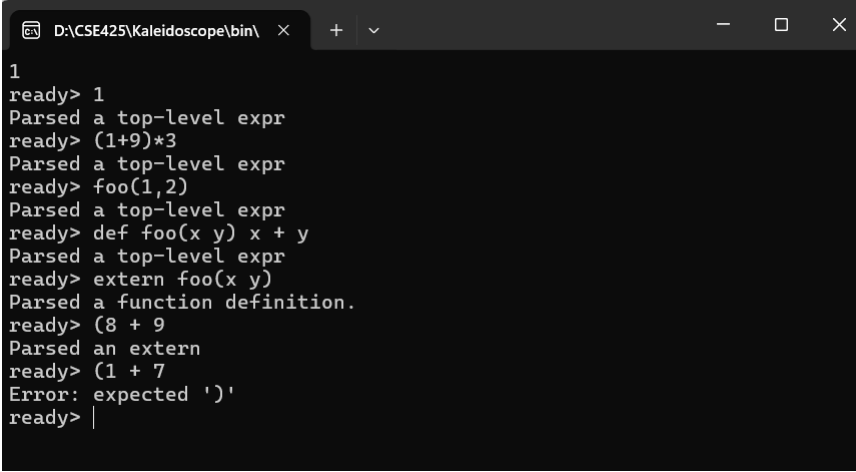
D:\CSE425\Kaleidoscope\bin\ x + - □ ×
Ready for input. Type Kaleidoscope code and press Enter:
Kaleidoscope code
Got token: -4 (Identifier: Kaleidoscope)
Got token: -4 (Identifier: code)
int 45
Got token: -4 (Identifier: int)
Got token: -5 (Number: 45)
if (a>b)
Got token: -4 (Identifier: if)
Got token: 40 (Symbol: '(')
Got token: -4 (Identifier: a)
Got token: 62 (Symbol: '>')
Got token: -4 (Identifier: b)
Got token: 41 (Symbol: ')')
|

```

Fig. 1. Overview of the lexer's tokenization process.

3.2 Parser and AST Construction

After the lexer has done its job of breaking the raw texts into lexemes and tokens, the parser converts it into an **Abstract Syntax Tree (AST)**. For this case, parsing combines two techniques: straightforward **recursive descent** for literals, parenthesized expressions, and function calls, each handled by its own parse routine operating on a single-token lookahead buffer (CurTok), and **operator-precedence parsing** for binary expressions. The design is very important as it cleanly separates from how it was written, providing later compiler stages with a simple, unambiguous structure to operate on.



```

D:\CSE425\Kaleidoscope\bin\ x + - □ ×
1
ready> 1
Parsed a top-level expr
ready> (1+9)*3
Parsed a top-level expr
ready> foo(1,2)
Parsed a top-level expr
ready> def foo(x y) x + y
Parsed a top-level expr
ready> extern foo(x y)
Parsed a function definition.
ready> (8 + 9
Parsed an extern
ready> (1 + 7
Error: expected ')'
ready> |

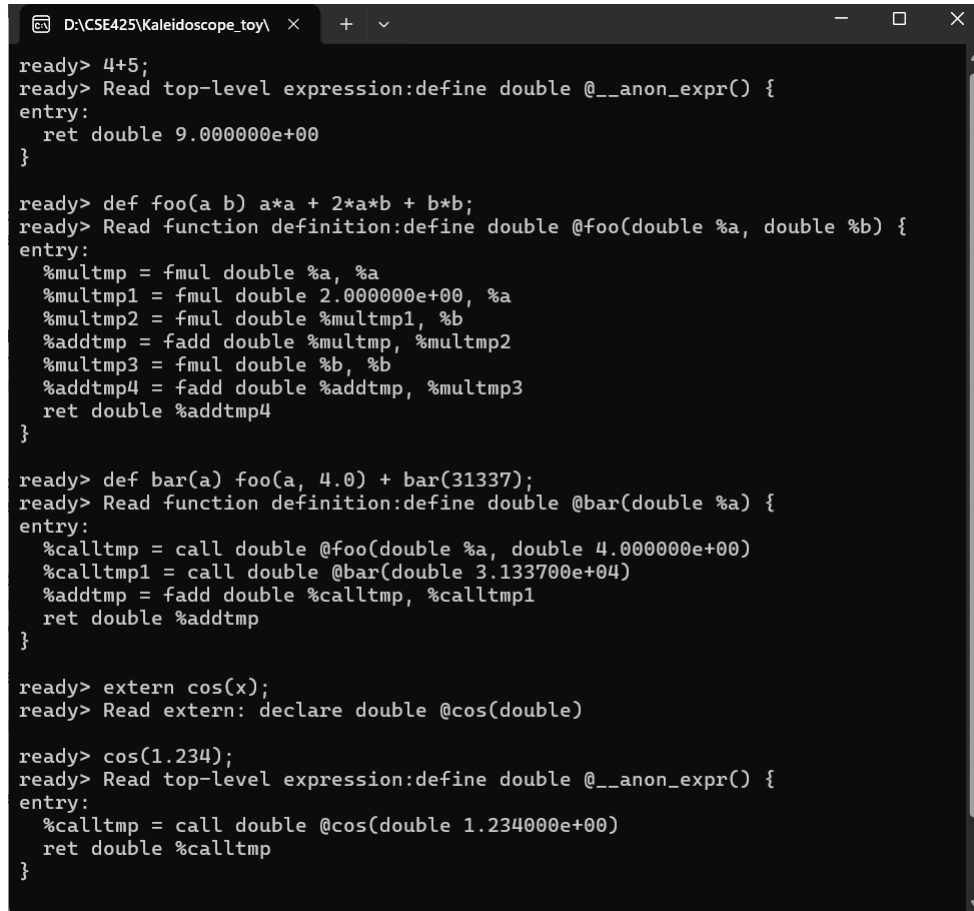
```

Fig. 2. Overview of the recursive-descent and operator-precedence parsing pipeline.

3.3 Code Generation with LLVM IR

With the AST built, each node is given a `codegen()` method that lowers it into **LLVM Intermediate Representation (IR)**, returning an LLVM `Value*` representing a Static Single Assignment (SSA) register.

Function prototypes and bodies construct an LLVM `FunctionType`, an entry `BasicBlock`, and finally verifying the result with `verifyFunction`. This stage is significant because it is where Kaleidoscope's abstract structure becomes concrete. At the end, the output IR is what all of LLVM's later optimization, JIT execution, and object-code emission stages consume.



```

ready> 4+5;
ready> Read top-level expression:define double @__anon_expr() {
entry:
    ret double 9.000000e+00
}

ready> def foo(a b) a*a + 2*a*b + b*b;
ready> Read function definition:define double @foo(double %a, double %b) {
entry:
    %multmp = fmul double %a, %a
    %multmp1 = fmul double 2.000000e+00, %a
    %multmp2 = fmul double %multmp1, %b
    %addtmp = fadd double %multmp, %multmp2
    %multmp3 = fmul double %b, %b
    %addtmp4 = fadd double %addtmp, %multmp3
    ret double %addtmp4
}

ready> def bar(a) foo(a, 4.0) + bar(31337);
ready> Read function definition:define double @bar(double %a) {
entry:
    %calltmp = call double @foo(double %a, double 4.000000e+00)
    %calltmp1 = call double @bar(double 3.133700e+04)
    %addtmp = fadd double %calltmp, %calltmp1
    ret double %addtmp
}

ready> extern cos(x);
ready> Read extern: declare double @cos(double)

ready> cos(1.234);
ready> Read top-level expression:define double @__anon_expr() {
entry:
    %calltmp = call double @cos(double 1.234000e+00)
    ret double %calltmp
}

```

Fig. 3. Overview of AST-to-LLVM IR Generation.

3.4 Optimizations And JIT Compilation

IR can't catch everything, that's why this chapter added two capabilities: **optimization passes** and **JIT Compiler**. A `FunctionPassManager` is set up with a handful of - `InstCombinePass`, `ReassociatePass`, `GVNPass`, and `SimplifyCFGPass`. This results in the disappearance of redundant work, and the IR gets noticeably leaner.

```

PS C:\kaleidoscope> ./toy
ready> def test(x) 1+2*x;
ready> Read function definition:define double @test(double %x) {
entry:
    %addtmp = fadd double %x, 3.000000e+00
    ret double %addtmp
}

ready> def test(x) (1+2*x)*(x+(1+2));
ready> Read function definition:define double @test(double %x) {
entry:
    %addtmp = fadd double %x, 3.000000e+00
    %multmp = fmul double %addtmp, %addtmp
    ret double %multmp
}

In KaleidoscopeJIT, duplicate definition of symbol 'test'
PS C:\kaleidoscope>

```

Fig. 4. Optimizations and JIT execution flow.

```

PS C:\kaleidoscope> ./toy
ready> 4+5;
ready> Evaluated to 9.000000
ready> def testfunc(x y) x + y*2;
ready> Read function definition:define double @testfunc(double %x,
double %y) {
entry:
    %multmp = fmul double %y, 2.000000e+00
    %addtmp = fadd double %x, %multmp
    ret double %addtmp
}

ready> testfunc(4, 10);
ready> Evaluated to 24.000000
ready> testfunc(5, 10);
ready> Evaluated to 25.000000
ready>

```

Fig. 5. Optimization pass pipeline.

3.5 Control Flow Statement

Up to this point, Kaleidoscope could do math and call functions, but not exactly a real programming language. That's why this chapter introduces two of the classics: *if/then/else* and the *for* loop and along the way.

If/then/else. Here's the twist: in Kaleidoscope, *everything* is an expression, not a statement and there's no such thing as a lonely *if* that just does something and shrugs. It has to *return* a value. The parser wraps this up in an *IfExprAST* holding a condition and two branches.

Both branches jump to a shared *MergeBB*, where a *phi* node picks the right value depending on which block execution came from.

The syntax of **For loop** is for *i* = start, cond, step in body, it becomes a *ForExprAST*. Codegen sets up a loop header with its own *phi* node for the induction variable, kicked off with the starting value.

The loop variable is temporarily added to *NamedValues* (shadowing any outer variable of the same name), the body is codegen'd, then the step is added and the end condition tested to decide whether to branch back to the loop or fall through to afterloop.

Lastly, rather than implementing a full SSA-construction algorithm in the frontend, the compiler inserts *phi* nodes directly by hand for these AST-driven control-flow merges.

```

PS C:\kaleidoscope> ./toy
ready> extern foo();
ready> Read extern: declare double @foo()

ready> extern bar();
ready> Read extern: declare double @bar()

ready> def baz(x) if x then foo() else bar();
ready> Read function definition:define double @baz(double %x) {
entry:
    %ifcond = fcmp ueq double %x, 0.000000e+00
    br i1 %ifcond, label %else, label %then

then:
    %calltmp = call double @foo()           ; preds = %entry
    br label %ifcont

else:
    %calltmp1 = call double @bar()          ; preds = %entry
    br label %ifcont

ifcont:
    %then
    %iftmp = phi double [ %calltmp, %then ], [ %calltmp1, %else ]
    ret double %iftmp
}

```

Fig. 6. Control Flow Statements-1.

```

ready> extern putchar(char);
ready> Read extern: declare double @putchar(double)

ready> def printstar(n)
ready>     for i = 1, i < n, 1.0 in
ready>         putchar(42);
Read function definition:define double @printstar(double %n) {
entry:
    br label %loop

loop:
    %entry
    %i = phi double [ 1.000000e+00, %entry ], [ %nextvar, %loop ]
    %calltmp = call double @putchar(double 4.200000e+01)
    %nextvar = fadd double %i, 1.000000e+00
    %cmptmp = fcmp ult double %i, %n
    br i1 %cmptmp, label %loop, label %afterloop

afterloop:
    ret double 0.000000e+00           ; preds = %loop
}

ready> printstar(20);
ready> *****Evaluated to 0.000000
ready> ready>

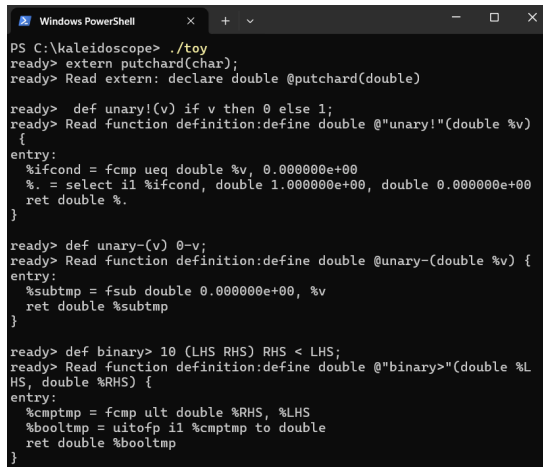
```

Fig. 7. Control Flow Statements-2

3.6 User-Defined Operators

This chapter extends Kaleidoscope with **user-defined operators**, letting programmers introduce new unary and binary operators (with a chosen precedence) entirely from within the language, rather than hard-coding them in the compiler.

Since the parser already uses operator-precedence parsing for binary expressions, new operators can be registered dynamically at runtime by inserting into the precedence table. A user-defined operator is really just an ordinary function whose name happens to be "binary%c" or "unary%c" (e.g. binary|), exploiting the fact that LLVM symbol names can contain arbitrary characters.



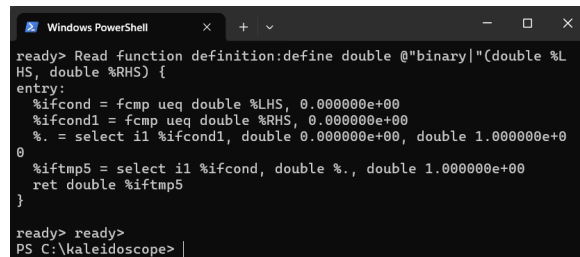
```
PS C:\kaleidoscope> ./toy
ready> extern putchar(char);
ready> Read extern: declare double @putchar(double)

ready> def unary!(v) if v then 0 else 1;
ready> Read function definition:define double @"unary!"(double %v)
{
entry:
  %ifcond = fcmp ueq double %v, 0.000000e+00
  %. = select i1 %ifcond, double 1.000000e+00, double 0.000000e+00
  ret double %.
}

ready> def unary-(v) 0-v;
ready> Read function definition:define double @unary-(double %v) {
entry:
  %subtmp = fsub double 0.000000e+00, %v
  ret double %subtmp
}

ready> def binary> 10 (LHS RHS) RHS < LHS;
ready> Read function definition:define double @"binary>"(double %L
HS, double %RHS) {
entry:
  %cmptmp = fcmp ult double %RHS, %LHS
  %booltmp = uitofp i1 %cmptmp to double
  ret double %booltmp
}
```

Fig. 8. User Defined Operators - 1.



```
ready> Read function definition:define double @"binary|"(double %L
HS, double %RHS) {
entry:
  %ifcond = fcmp ueq double %LHS, 0.000000e+00
  %ifcond1 = fcmp ueq double %RHS, 0.000000e+00
  %. = select i1 %ifcond1, double 0.000000e+00, double 1.000000e+0
0
  %iftmp5 = select i1 %ifcond, double %, double 1.000000e+00
  ret double %iftmp5
}

ready> ready>
PS C:\kaleidoscope> |
```

Fig. 9. User Defined Operators - 2.

3.7 Mutable Variables

This chapter finally lets variables change, adding the = assignment operator and a var/in construct for declaring locals. It shows how LLVM's *mem2reg* pass makes this possible *without* the front-end having to build SSA form itself. LLVM requires SSA registers but not SSA memory. Each mutable variable is instead given a stack slot via *alloca*, with reads/writes becoming plain load/ store instructions.

Put it all together, and Kaleidoscope stops being a purely math-expression toy and starts looking like a real imperative language (rather than recursive), closing the gap between it and a conventional language.

```

PS C:\kaleidoscope> ./toy
ready> extern printf(x);
ready> Read extern: declare double @printf(double)

ready> def binary : 1 (x y) y;
ready> Read function definition:define double @"binary:"(double %x
, double %y) {
entry:
  ret double %y
}

ready> def test(x)
ready>      printf(x) :
      x = 4 :
      printf(x);
Read function definition:define double @test(double %x) {
entry:
  %calltmp = call double @printf(double %x)
  %binop = call double @"binary:"(double %calltmp, double 4.000000
e+00)
  %calltmp4 = call double @printf(double 4.000000e+00)
  %binop5 = call double @"binary:"(double %binop, double %calltmp4
)
  ret double %binop5
}

ready> test(123);
ready> 123.000000
4.000000
Evaluated to 0.000000
ready>

```

Fig. 10. Overview of Mutable Variables.

3.8 Object File

This chapter moves Kaleidoscope away from JIT execution and towards producing a standalone **object file** that can be linked into an ordinary C/C++ program, using LLVM's target-machine and code-emission APIs. The emitted object file exports ordinary C-callable symbols (e.g. an average function), so it can be linked directly against a separate `main.cpp` with a matching `extern "C"` declaration, producing a normal native executable.

```

PS C:\kaleidoscope> ./toy
ready> def average(x y) (x + y) * 0.5;^Z^Z
Read function definition:define double @average(double %x, double
%y) {
entry:
  %y2 = alloca double, align 8
  %x1 = alloca double, align 8
  store double %x, ptr %x1, align 8
  store double %y, ptr %y2, align 8
  %x3 = load double, ptr %x1, align 8
  %y4 = load double, ptr %y2, align 8
  %addtmp = fadd double %x3, %y4
  %multmp = fmul double %addtmp, 5.000000e-01
  ret double %multmp
}
^Z
Error: unknown token when expecting an expression
Wrote output.o
PS C:\kaleidoscope> & "C:\\llvm-project\\build\\bin\\clang++.exe" main
.cpp output.o -o main.exe
PS C:\kaleidoscope> ./main.exe
average of 3.0 and 4.0: 3.5
PS C:\kaleidoscope>

```

Fig. 11. Compiling to Object Code - 1.

```

// main.cpp
#include <iostream>

extern "C" {
    double average(double, double);
}

int main() {
    std::cout << "average of 3.0 and 4.0: " << average(3.0, 4.0) << std::endl;
}

```

Fig. 12. Compiling to Object Code - 2

3.9 Adding Debug Information

This chapter adds DWARF debug information to Kaleidoscope, enabling breakpoints, variable inspection, and backtraces in gdb. A `DIBuilder` constructs compile-unit and function metadata, while a new `SourceLocation` struct tracks line/column info through the lexer and AST. Each function gets a `DISubprogram`, each argument a `DILocalVariable`, letting compiled Kaleidoscope programs debug like ordinary C code.

4 Additional Features

For the extension to “Kaleidoscope”, a while loop was added, following the pattern suggested in the whole tutorial so far (lexer to AST to parser to codegen).

A new token tok_while is recognized by the lexer, a WhileExprAST node stores the condition and body, and ParseWhileExpr() parses the while cond in body syntax.

Codegen builds three basic blocks – condition, loop body, and afterloop and then it rechecks the condition each iteration.

Unlike for, while has no induction variable, so no PHI node or alloca is needed for loop control, it just needs the condition recheck and a backward branch after the body runs. Combined with Chapter 7’s mutable variables, this enables imperative loops such as a countdown function that mutates and tests a variable on each pass.

```
enum Token {
    tok_eof = -1,

    // commands
    tok_def = -2,
    tokExtern = -3,

    // primary
    tok_identifier = -4,
    tok_number = -5,

    // control
    tok_if = -6,
    tok_then = -7,
    tok_else = -8,
    tok_for = -9,
    tok_in = -10,

    // operators
    tok_binary = -11,
    tok_unary = -12,

    // var definition
    tok_var = -13,

    // while definition
    tok_while = -14
};
```

Fig. 13. Overview of How While Loop Was Added - 1.

```
/// WhileExprAST - Expression class for while/in.
class WhileExprAST : public ExprAST {
    std::unique_ptr<ExprAST> Cond, Body;

public:
    WhileExprAST(std::unique_ptr<ExprAST> Cond, std::unique_ptr<ExprAST> Body)
        : Cond(std::move(Cond)), Body(std::move(Body)) {}

    Value *codegen() override;
};
```

Fig. 15. Overview of How While Loop Was Added - 3.

```
/// gettok - Return the next token from standard input.
static int gettok() {
    static int lastChar = ' ';

    // Skip any whitespace.
    while (isspace(lastChar))
        lastChar = getchar();

    if (!isalpha(lastChar)) { // Identifier: [a-zA-Z][a-zA-Z0-9]*
        identifierStr = lastChar;
        while (isalnum(lastChar - getchar()))
            identifierStr += lastChar;

        if (identifierStr == "def")
            return tok_def;
        if (identifierStr == "extern")
            return tok_extern;
        if (identifierStr == "if")
            return tok_if;
        if (identifierStr == "then")
            return tok_then;
        if (identifierStr == "else")
            return tok_else;
        if (identifierStr == "for")
            return tok_for;
        if (identifierStr == "in")
            return tok_in;
        if (identifierStr == "binary")
            return tok_binary;
        if (identifierStr == "unary")
            return tok_unary;
        if (identifierStr == "var")
            return tok_var;
        if (identifierStr == "while")
            return tok_while;
        return tok_identifier;
    }
}
```

Fig. 14. Overview of How While Loop Was Added - 2.

```
/// whileExpr ::= 'while' expression 'in' expression
static std::unique_ptr<ExprAST> ParseWhileExpr() {
    getNextToken(); // eat the while.

    auto Cond = ParseExpression();
    if (!Cond)
        return nullptr;

    if (CurTok != tok_in)
        return LogError("expected 'in' after while condition");
    getNextToken(); // eat 'in'.

    auto Body = ParseExpression();
    if (!Body)
        return nullptr;

    return std::make_unique<WhileExprAST>(std::move(Cond), std::move(Body));
}
```

Fig. 16. Overview of How While Loop Was Added - 4.

```
static std::unique_ptr<ExprAST> ParsePrimary() {
    switch (CurTok) {
    default:
        return LogError("unknown token when expecting an expression");
    case tok_identifier:
        return ParseIdentifierExpr();
    case tok_number:
        return ParseNumberExpr();
    case '(':
        return ParseParenExpr();
    case tok_if:
        return ParseIfExpr();
    case tok_for:
        return ParseForExpr();
    case tok_var:
        return ParseVarExpr();
    case tok_while:
        return ParseWhileExpr();
    }
}
```

Fig. 17. Overview of How While Loop Was Added - 5.

```
Value *WhileExprAST::codegen() {
    Function *TheFunction = Builder->GetInsertBlock()->getParent();

    BasicBlock *CondBB = BasicBlock::Create(*TheContext, "whilecond",
    TheFunction);
    BasicBlock *LoopBB = BasicBlock::Create(*TheContext, "whileloop",
    TheFunction);
    BasicBlock *AfterBB = BasicBlock::Create(*TheContext, "afterwhile",
    TheFunction);

    Builder->CreateBr(CondBB);
    Builder->SetInsertPoint(CondBB);

    Value *CondV = cond->codegen();
    if (!CondV)
        return nullptr;
    CondV = Builder->CreateCmpONE(
        CondV, ConstantFP::get(*TheContext, APFloat(0.0)), "whilecond");
    Builder->CreateCondBr(CondV, LoopBB, AfterBB);

    Builder->SetInsertPoint(LoopBB);
    if (!Body->codegen())
        return nullptr;
    Builder->CreateBr(CondBB); // re-check condition

    Builder->SetInsertPoint(AfterBB);

    // while always returns 0.0, matching the convention used by 'for'.
    return Constant::getNullValue(Type::getDoubleTy(*TheContext));
}
```

Fig. 18. Overview of How While Loop Was Added - 6.

```
PS C:\kaleidoscope> ./toy
ready> extern printf(x);
ready> Read extern: declare double @printf(double)

ready> def binary : 1 (x y) y;
ready> Read function definition:define double @"binary:"(double %x
, double %y) {
entry:
    ret double %y
}

ready> def countdown(n)
ready>      var i = n in
```

Fig. 19. Overview of How While Loop Was Added - 7.

```
Windows PowerShell
ready> def countdown(n)
ready>      var i = n in
    (while 0 < i in
        printf(i) :
        i = i - 1) :
    0;

Read function definition:define double @countdown(double %n) {
entry:
    br label %whilecond

whilecond:
    ; preds = %while
    loop, %entry
    %i0 = phi double [ %n, %entry ], [ %subtmp, %whileloop ]
    %cmtmp = fcmp ugt double %i0, 0.000000e+00
    br i1 %cmtmp, label %whileloop, label %afterwhile

whileloop:
    ; preds = %while
    cond
    %calltmp = call double @printf(double %i0)
    %subtmp = fadd double %i0, -1.000000e+00
    %binop = call double @"binary:"(double %calltmp, double %subtmp)
    br label %whilecond

afterwhile:
    ; preds = %while
    cond
    %binop7 = call double @"binary:"(double 0.000000e+00, double 0.0
00000e+00)
    ret double %binop7
}

ready> countdown(5);
ready> 5.000000
4.000000
3.000000
2.000000
1.000000
Evaluated to 0.000000
ready> |
```

Fig. 20. Overview of How While Loop Was Added - 8.

5 Conclusion

In this project, we built “Kaleidoscope”, which is a small compiler with more power than its size might suggest, entirely from scratch using LLVM. What began as a simple expression evaluator gradually evolved into a complete compiler with a handwritten lexer, recursive-descent parser, custom AST, and LLVM IR generator. We also went beyond interpretation by supporting both Just-In-Time (JIT) execution and Ahead-Of-Time (AOT) object file generation.

Throughout the project, compiler theory became practical. We tackled challenges such as operator precedence and SSA-based variables while taking advantage of LLVM features like basic blocks, ϕ (phi) nodes, and automatic optimization passes.

The language was further extended with:

- **Control Flow:** if/then/else statements and for loops.
- **Custom Operators:** User-defined unary and binary operators with adjustable precedence.
- **Local State:** Mutable variables using the var/in construct.
- **Tooling & Debugging:** Native target-machine code generation and DWARF debug metadata through DIBuilder, allowing the generated code to be inspected using debuggers such as gdb and lldb.

Overall, the project showed that building a programming language does not mean reinventing everything from scratch. With a well-designed front-end and LLVM doing the heavy lifting in the back-end, even a small compiler can pack quite a punch.

References

- [1] LLVM Project. 2026. *My First Language Frontend with LLVM Tutorial*. <https://llvm.org/docs/tutorial/MyFirstLanguageFrontend/index.html>